Capstone Project: Fair Value Forecasting Syst for Commodity Portfolio

Course: Fair Value Modelling for Commodities Trading
Duration: 4 - 6 weeks
Points: 100

Project Overview

In this capstone project, you will build a complete, production-ready fair value forecasting and sig generation system for a commodity portfolio. This project integrates all concepts from the course

- Point-in-time data infrastructure
- Temporal feature engineering
- Multiple fair value models
- Walk-forward backtesting
- Signal generation and risk management

Learning Objectives

By completing this project, you will demonstrate ability to:

1. Build robust temporal data pipelines avoiding look-ahead bias
2. Implement and compare multiple fair value modeling approaches
3. Validate models using proper walk-forward methodology
4. Generate actionable trading signals with risk controls
5. Present findings in a professional, reproducible manner

Deliverables

1. **This completed notebook** with all TODO sections implemented
2. **Data pipeline** with point-in-time database
3. **Model comparison report** with validation metrics
4. **Signal backtesting results** with performance attribution
5. **Executive summary** (500-1000 words)

Grading Rubric (100 points)

Component	Points	Criteria
Part 1: Data Pipeline	20	- Proper point-in-time structure (8 pts) - Data quality documentation (6 pts) - Multiple data sources (6 pts)
Part 2: Exploratory Analysis	15	

Component	Points	Criteria
		- Temporal EDA adherence (7 pts) - Insightful visualizations (5 pts) - Statistical analysis (3 pts)
Part 3 : Fair Value Models	25	- Multiple model types (10 pts) - Justified parameter choices (8 pts) - Clear documentation (7 pts)
Part 4 : Walk-Forward Validation	20	- Proper fold structure (8 pts) - No look-ahead bias (8 pts) - Comprehensive metrics (4 pts)
Part 5 : Signal Generation	15	- Signal logic (6 pts) - Risk management (5 pts) - Performance metrics (4 pts)
Part 6 : Executive Report	5	- Clear communication (3 pts) - Professional presentation (2 pts)

Submission Checklist

Before submitting, verify:

- ☐ All TODO sections completed
- ☐ All code cells execute without errors (Kernel → Restart & Run All)
- ☐ Data sources documented with access instructions
- ☐ No look-ahead bias in validation (verified with `detect_data_leakage()`)
- ☐ All visualizations have clear titles and labels
- ☐ Executive summary completed
- ☐ Point-in-time database file included in submission
- ☐ Model artifacts saved to disk

Commodity Selection Guidelines

Choose 3 - 5 **related commodities** from one of these complexes:

Energy Complex (Recommended)

- WTI Crude Oil
- Brent Crude Oil
- Natural Gas
- RBOB Gasoline
- Heating Oil

Why: Rich fundamental data (EIA inventories), clear supply/demand drivers

Metals Complex

- Gold
- Silver
- Copper
- Platinum
- Palladium

Why: Macro drivers, industrial vs precious metals dynamics

Agriculture Complex

- Corn
- Wheat
- Soybeans
- Soybean Oil
- Soybean Meal

Why: Strong seasonality, weather impacts, crush spreads

Project Timeline

Week 1 - 2 : Data Pipeline & EDA (Parts 1 - 2)
Week 3 - 4 : Model Development & Validation (Parts 3 - 4)
Week 5 : Signal Generation (Part 5)
Week 6 : Final Report & Polishing (Part 6)

Setup & Configuration

In [1]:
```
# Standard imports
import sys
import os
from pathlib import Path
import warnings
warnings.filterwarnings('ignore')

# Data handling
import numpy as np
import pandas as pd
from datetime import datetime, timedelta
import sqlite3

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

```python
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

# Set style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
%matplotlib inline

# Add src to path
project_root = Path.cwd().parent
sys.path.insert(0, str(project_root / 'src'))

# Fair value toolkit imports
from fair_value_toolkit import (
    # Point-in-time data
    PointInTimeRecord,
    PointInTimeDataFrame,
    PointInTimeDatabase,
    # Models
    FairValueModel,
    InventoryFairValueModel,
    CostOfCarryModel,
    SupplyDemandModel,
    CostCurveModel,
    EnsembleFairValueModel,
    # Validation
    WalkForwardValidator,
    TimeSeriesSplit,
    detect_data_leakage,
    calculate_model_decay,
    # Signals
    SignalGenerator,
    FairValueSignal,
    PositionSizer,
    RiskManager,
    # Features
    TemporalFeatureEngineer,
    create_lagged_features,
    create_rolling_features,
    create_seasonal_features,
)

print("Setup complete!")
print(f"Project root: {project_root}")
```

```
Setup complete!
Project root: /home/user/claude-template/fair-value-commodities-course
```

In [2]:

```python
# Configuration
CONFIG = {
    # Data paths
    'data_dir': project_root / 'data' / 'capstone',
    'database_path': project_root / 'data' / 'capstone' / 'pit_database.d
    'models_dir': project_root / 'data' / 'capstone' / 'models',
    'results_dir': project_root / 'data' / 'capstone' / 'results',

    # Date ranges
    'start_date': '2018-01-01',  # Adjust as needed
    'end_date': '2024-11-30',
```

```
    'train_start': '2018-01-01',
    'validation_start': '2022-01-01',
    'test_start': '2023-06-01',

    # Walk-forward parameters
    'initial_train_window': 252 * 3,  # 3 years
    'validation_window': 126,  # 6 months
    'test_window': 63,  # 3 months
    'gap_days': 5,  # Gap between train and validation

    # Signal parameters
    'entry_threshold': 1.5,  # Entry z-score
    'exit_threshold': 0.5,   # Exit z-score
    'max_position_size': 0.2,  # 20% max per commodity
    'portfolio_leverage': 1.0,

    # Model parameters
    'confidence_level': 0.95,
    'ensemble_weights': None,  # Will be optimized
}

# Create directories
CONFIG['data_dir'].mkdir(parents=True, exist_ok=True)
CONFIG['models_dir'].mkdir(parents=True, exist_ok=True)
CONFIG['results_dir'].mkdir(parents=True, exist_ok=True)

print("Configuration loaded:")
for key, value in CONFIG.items():
    print(f"  {key}: {value}")
```

```
Configuration loaded:
  data_dir: /home/user/claude-template/fair-value-commodities-course/data/cap
  database_path: /home/user/claude-template/fair-value-commodities-course/dat
capstone/pit_database.db
  models_dir: /home/user/claude-template/fair-value-commodities-course/data/
capstone/models
  results_dir: /home/user/claude-template/fair-value-commodities-course/data/
capstone/results
  start_date: 2018-01-01
  end_date: 2024-11-30
  train_start: 2018-01-01
  validation_start: 2022-01-01
  test_start: 2023-06-01
  initial_train_window: 756
  validation_window: 126
  test_window: 63
  gap_days: 5
  entry_threshold: 1.5
  exit_threshold: 0.5
  max_position_size: 0.2
  portfolio_leverage: 1.0
  confidence_level: 0.95
  ensemble_weights: None
```

Part 1 : Data Pipeline (2 0 points)

Build a point-in-time data infrastructure for your commodity portfolio. This section focuses on cre
robust data pipeline that respects temporal boundaries.

Requirements

1. **Select** 3 - 5 **commodities** from the same complex
2. **Download data** from at least 2 different sources:
    - Yahoo Finance for prices
    - FRED for inventories/economic indicators
    - EIA for energy fundamentals (if applicable)
3. **Build point-in-time database** with proper temporal metadata
4. **Document data quality**: missing values, outliers, publication lags

Grading Criteria

- Point-in-time structure with observation/publication dates (8 pts)
- Data quality documentation (6 pts)
- Multiple data sources (6 pts)

1.1 Commodity Selection

TODO: Document your commodity selection and justification.

Selected Commodities

TODO: Fill in your selections

Complex: [Energy/Metals/Agriculture]

Commodities:

1. [Commodity 1] - [Ticker Symbol]
2. [Commodity 2] - [Ticker Symbol]
3. [Commodity 3] - [Ticker Symbol]
4. [Commodity 4] - [Ticker Symbol] (optional)
5. [Commodity 5] - [Ticker Symbol] (optional)

Justification: [Explain why you selected these commodities. Consider:

- Market liquidity and data availability
- Economic relationships between commodities
- Interesting spread or relative value opportunities
- Data quality and fundamental driver visibility]

In [3]:
```
# TODO: Define your commodity universe
COMMODITIES = {
    'commodity_1': {
        'name': 'TODO',
        'ticker': 'TODO',  # Yahoo Finance ticker
        'fred_series': ['TODO'],  # FRED series IDs for fundamentals
        'unit': 'TODO',  # e.g., 'USD/bbl', 'USD/oz'
```

```
    },
    # Add more commodities...
}

print(f"Selected {len(COMMODITIES)} commodities:")
for key, info in COMMODITIES.items():
    print(f"  {info['name']} ({info['ticker']})")
```

```
Selected 1 commodities:
  TODO (TODO)
```

1.2 Data Collection

Download price and fundamental data from multiple sources.

In [4]:
```
# TODO: Install data fetchers if needed
# !pip install yfinance pandas-datareader fredapi

import yfinance as yf
import pandas_datareader as pdr
from pandas_datareader import data as web

print("Data fetching libraries imported")
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[4], line 4
      1 # TODO: Install data fetchers if needed
      2 # !pip install yfinance pandas-datareader fredapi
----> 4 import yfinance as yf
      5 import pandas_datareader as pdr
      6 from pandas_datareader import data as web

ModuleNotFoundError: No module named 'yfinance'
```

In [5]:
```
# TODO: Download price data from Yahoo Finance
def download_price_data(ticker, start_date, end_date):
    """
    Download price data from Yahoo Finance.

    Parameters
    ----------
    ticker : str
        Yahoo Finance ticker symbol
    start_date : str
        Start date in 'YYYY-MM-DD' format
    end_date : str
        End date in 'YYYY-MM-DD' format

    Returns
    -------
    pd.DataFrame
        Price data with columns: Open, High, Low, Close, Volume, Adj Clos
    """
    # TODO: Implement data download
    # Hint: Use yf.download()
    # Remember to handle missing data and convert to point-in-time format
    pass

# Test the function
```

```
# sample_data = download_price_data('CL=F', CONFIG['start_date'], CONFIG[
# print(sample_data.head())
```

In [6]:
```
# TODO: Download fundamental data from FRED
def download_fred_data(series_id, start_date, end_date):
    """
    Download fundamental data from FRED.

    Parameters
    ----------
    series_id : str
        FRED series identifier (e.g., 'DCOILWTICO' for WTI crude)
    start_date : str
        Start date in 'YYYY-MM-DD' format
    end_date : str
        End date in 'YYYY-MM-DD' format

    Returns
    -------
    pd.DataFrame
        Fundamental data
    """
    # TODO: Implement FRED data download
    # Hint: Use pdr.DataReader(series_id, 'fred', start_date, end_date)
    # Note: FRED data publication lags - document this!
    pass

# Test the function
# sample_fred = download_fred_data('DCOILWTICO', CONFIG['start_date'], CO
# print(sample_fred.head())
```

1.3 Point-in-Time Database Construction

Convert raw data to point-in-time format with proper temporal metadata.

In [7]:
```
# TODO: Create PointInTimeDatabase
pit_db = PointInTimeDatabase(str(CONFIG['database_path']))

print(f"Point-in-time database created at: {CONFIG['database_path']}")
```

```
Point-in-time database created at: /home/user/claude-template/fair-value-
commodities-course/data/capstone/pit_database.db
```

In [8]:
```
# TODO: Convert price data to point-in-time records
def price_data_to_pit_records(df, series_id, commodity_name):
    """
    Convert price DataFrame to PointInTimeRecord objects.

    Key consideration: For daily prices, observation_date = publication_d
    since prices are known at market close.

    Parameters
    ----------
    df : pd.DataFrame
        Price data with datetime index
    series_id : str
        Unique series identifier
    commodity_name : str
        Human-readable commodity name
```

```
    Returns
    -------
    List[PointInTimeRecord]
        List of temporal records
    """
    records = []

    # TODO: Implement conversion
    # Hint: Iterate through DataFrame rows
    # Create PointInTimeRecord for each observation
    # Set observation_date = index, publication_date = index (for prices)
    # Use appropriate metadata

    return records

print("Price to PIT converter ready")
```

```
Price to PIT converter ready
```

In [9]:

```
# TODO: Convert fundamental data to point-in-time records
def fundamental_data_to_pit_records(df, series_id, publication_lag_days=2
    """
    Convert fundamental DataFrame to PointInTimeRecord objects.

    Key consideration: Fundamental data (inventories, production) typical
    has a publication lag. For example, EIA petroleum status reports are
    published on Wednesday for data through the previous Friday.

    Parameters
    ----------
    df : pd.DataFrame
        Fundamental data with datetime index
    series_id : str
        Unique series identifier
    publication_lag_days : int
        Days between observation and publication

    Returns
    -------
    List[PointInTimeRecord]
        List of temporal records with proper publication dates
    """
    records = []

    # TODO: Implement conversion
    # Hint: Add publication_lag_days to observation_date
    # Document the lag for each data source
    # Consider different lags for different series

    return records

print("Fundamental to PIT converter ready")
```

```
Fundamental to PIT converter ready
```

In [10]:

```
# TODO: Download and store all data
print("Downloading and storing data...\n")

for commodity_key, commodity_info in COMMODITIES.items():
    print(f"Processing {commodity_info['name']}...")
```

```
    # TODO: Download price data
    # price_df = download_price_data(...)

    # TODO: Convert to PIT records
    # price_records = price_data_to_pit_records(...)

    # TODO: Insert into database
    # pit_db.insert_records(price_records)

    # TODO: Download fundamental data
    # for series_id in commodity_info['fred_series']:
    #     fred_df = download_fred_data(...)
    #     fred_records = fundamental_data_to_pit_records(...)
    #     pit_db.insert_records(fred_records)

    print(f"  ✓ {commodity_info['name']} complete")

print("\nData pipeline complete!")
```

```
Downloading and storing data...

Processing TODO...
  ✓ TODO complete

Data pipeline complete!
```

1.4 Data Quality Assessment

Document data quality issues and characteristics.

In [11]:
```
# TODO: Analyze data quality
def assess_data_quality(series_id, as_of_date=None):
    """
    Generate data quality report for a series.

    Returns
    -------
    dict
        Quality metrics: completeness, outliers, gaps, etc.
    """
    # TODO: Implement quality assessment
    # Consider:
    # - Missing value percentage
    # - Outlier detection (e.g., > 3 std dev)
    # - Maximum gap duration
    # - Data frequency consistency
    # - Publication lag statistics

    quality_report = {
        'series_id': series_id,
        'total_records': None,  # TODO
        'date_range': (None, None),  # TODO
        'missing_pct': None,  # TODO
        'outlier_count': None,  # TODO
        'max_gap_days': None,  # TODO
        'avg_publication_lag': None,  # TODO
    }
```

```
    return quality_report

print("Data quality assessor ready")
```

```
Data quality assessor ready
```

In [12]:
```
# TODO: Generate quality reports for all series
quality_reports = []

# for series in pit_db.list_series():
#     report = assess_data_quality(series)
#     quality_reports.append(report)

# Convert to DataFrame for easy viewing
# quality_df = pd.DataFrame(quality_reports)
# print(quality_df)
```

1.5 Data Quality Documentation

TODO: Summarize data quality findings

Data Quality Summary

TODO: Fill in your findings

Data Sources

Source	Series Count	Date Range	Frequency
Yahoo Finance	TODO	TODO	Daily
FRED	TODO	TODO	TODO
Other	TODO	TODO	TODO

Quality Issues

1. **Missing Data**: [Describe which series have gaps and why]
2. **Outliers**: [Document any suspicious values]
3. **Publication Lags**: [Document typical lags for fundamental data]
4. **Revisions**: [Note if any series undergo frequent revisions]

Data Treatment Decisions

- [How did you handle missing values?]
- [Did you filter any outliers? On what basis?]
- [How did you estimate publication dates for historical data?]

Part 2: Exploratory Data Analysis (15 points)

Perform exploratory analysis that respects temporal boundaries. All analysis must use `as_of` c
to avoid look-ahead bias.

Requirements

1. **Time series visualization** with point-in-time constraints
2. **Correlation analysis** across rolling windows
3. **Seasonality detection** and decomposition
4. **Data vintage comparison** showing impact of revisions
5. **Structural break analysis**

Grading Criteria

- Temporal EDA adherence (7 pts)
- Insightful visualizations (5 pts)
- Statistical analysis (3 pts)

2.1 Time Series Visualization

In [13]:
```
# TODO: Create temporal price visualization
def plot_commodity_prices(as_of_date=None):
    """
    Plot commodity prices as they were known at a specific date.

    Parameters
    ----------
    as_of_date : datetime or str, optional
        Date for point-in-time query. If None, use latest data.
    """
    fig, axes = plt.subplots(len(COMMODITIES), 1,
                             figsize=(15, 4*len(COMMODITIES)),
                             sharex=True)

    # TODO: Implement plotting
    # Hint: Use pit_db.query_series(..., as_of_date=as_of_date)
    # Plot each commodity on separate subplot
    # Add title indicating as_of_date

    plt.tight_layout()
    return fig

# Example usage:
# fig = plot_commodity_prices(as_of_date='2023-12-31')
# plt.show()
```

2.2 Correlation Analysis

In [14]:
```
# TODO: Calculate rolling correlations
def calculate_rolling_correlations(window_days=252, as_of_date=None):
    """
    Calculate rolling correlations between commodities.

    Parameters
    ----------
    window_days : int
        Rolling window size in days
```

```
    as_of_date : datetime or str, optional
        Point-in-time query date

    Returns
    -------
    pd.DataFrame
        Rolling correlation matrix
    """
    # TODO: Implement rolling correlation
    # Steps:
    # 1. Query price data for all commodities (as_of_date)
    # 2. Align to common dates
    # 3. Calculate returns
    # 4. Compute rolling correlation
    pass

# TODO: Visualize correlations
# rolling_corr = calculate_rolling_correlations(window_days=252)
# Plot heatmap or time series of correlations
```

2.3 Seasonality Analysis

In [15]:
```
# TODO: Detect and visualize seasonality
from statsmodels.tsa.seasonal import seasonal_decompose

def analyze_seasonality(series_id, as_of_date=None):
    """
    Perform seasonal decomposition and visualize components.

    Parameters
    ----------
    series_id : str
        Series to analyze
    as_of_date : datetime or str, optional
        Point-in-time query date
    """
    # TODO: Implement seasonality analysis
    # Steps:
    # 1. Query data (as_of_date)
    # 2. Perform seasonal decomposition
    # 3. Plot: original, trend, seasonal, residual
    # 4. Calculate seasonal strength metrics
    pass

# Apply to each commodity
# for commodity in COMMODITIES.values():
#     analyze_seasonality(f"{commodity['name']}.price")
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[15], line 2
      1 # TODO: Detect and visualize seasonality
----> 2 from statsmodels.tsa.seasonal import seasonal_decompose
      4 def analyze_seasonality(series_id, as_of_date=None):
      5     """
      6     Perform seasonal decomposition and visualize components.
      7
   (...)     13         Point-in-time query date
     14     """

ModuleNotFoundError: No module named 'statsmodels'
```

2.4 Data Vintage Comparison

Compare data as it was known at different points in time to understand revision impact.

In [16]:
```
# TODO: Compare data vintages
def compare_data_vintages(series_id, vintage_dates, observation_date):
    """
    Compare how a single observation changed across different vintages.

    Parameters
    ----------
    series_id : str
        Series to analyze
    vintage_dates : List[datetime]
        Dates representing different data vintages
    observation_date : datetime
        The observation date to track across vintages

    Returns
    -------
    pd.DataFrame
        Vintage comparison showing value evolution
    """
    # TODO: Implement vintage comparison
    # For each vintage_date:
    #   Query series as_of that date
    #   Extract value for observation_date
    # Show how the value changed over time
    pass

# Example: Track how an inventory report was revised
# vintage_comparison = compare_data_vintages(
#     series_id='EIA.CRUDE.STOCKS',
#     vintage_dates=['2024-01-17', '2024-01-24', '2024-01-31'],
#     observation_date='2024-01-12'
# )
```

2.5 EDA Insights Summary

TODO: Summarize key findings from exploratory analysis

Key Findings

TODO: Document your insights

Price Dynamics

- [Describe trend, volatility, regime changes]
- [Note any structural breaks or events]

Inter-Commodity Relationships

- [Describe correlation patterns]
- [Identify co-movements or divergences]

Seasonality

- [Document seasonal patterns by commodity]
- [Explain economic rationale for seasonality]

Data Revision Impact

- [Quantify typical revision magnitudes]
- [Assess impact on model inputs]

Implications for Modeling

- [What features look promising?]
- [What relationships should models capture?]
- [What data quality concerns need addressing?]

Part 3 : Fair Value Models (25 points)

Implement at least 2 different fair value models with clear economic justification.

Requirements

1. **Select 2-3 model types** appropriate for your commodities:
    - InventoryFairValueModel (storage theory)
    - CostOfCarryModel (futures pricing)
    - SupplyDemandModel (fundamental balance)
    - CostCurveModel (marginal producer)
2. **Feature engineering** with temporal discipline
3. **Parameter selection** with economic justification
4. **Model documentation** explaining assumptions

Grading Criteria

- Multiple model types (10 pts)
- Justified parameter choices (8 pts)
- Clear documentation (7 pts)

3.1 Feature Engineering

Create features that respect temporal boundaries.

In [17]:
```
# TODO: Build feature matrix for each commodity
def build_feature_matrix(commodity_key, as_of_date=None):
    """
    Create feature matrix for fair value modeling.

    Parameters
    ----------
    commodity_key : str
        Commodity identifier
    as_of_date : datetime or str, optional
        Point-in-time constraint

    Returns
    -------
    pd.DataFrame
        Feature matrix with temporal features
    """
    commodity_info = COMMODITIES[commodity_key]

    # TODO: Query base data
    # price_data = pit_db.query_series(..., as_of_date=as_of_date)
    # fundamental_data = pit_db.query_series(..., as_of_date=as_of_date)

    # TODO: Create lagged features
    # Use create_lagged_features() from toolkit

    # TODO: Create rolling features
    # Use create_rolling_features() from toolkit

    # TODO: Create seasonal features
    # Use create_seasonal_features() from toolkit

    # TODO: Create interaction features
    # e.g., inventory deviation * seasonality

    # TODO: Create relative features
    # e.g., z-scores, percentile ranks

    return None  # TODO: return feature DataFrame

print("Feature engineering function ready")
```

```
Feature engineering function ready
```

In [18]:
```
# TODO: Generate features for all commodities
feature_matrices = {}
```

```
# for commodity_key in COMMODITIES.keys():
#     features = build_feature_matrix(commodity_key)
#     feature_matrices[commodity_key] = features
#     print(f"Features for {COMMODITIES[commodity_key]['name']}: {feature
```

3.2 Model 1: Inventory-Based Fair Value

Implement inventory-based model using storage theory.

Model 1 Overview

TODO: Document your model choice and rationale

Model Type: [e.g., InventoryFairValueModel]

Economic Rationale: [Explain why this model is appropriate for your commodities. Reference s theory, convenience yield, etc.]

Key Features:

1. [Feature 1] - [Why included]
2. [Feature 2] - [Why included]
3. [Feature 3] - [Why included]

Model Assumptions:

- [List key assumptions]
- [Note any limitations]

In [19]:
```
# TODO: Initialize and configure Model 1
model_1_configs = {}

for commodity_key in COMMODITIES.keys():
    # TODO: Configure model parameters
    # Example:
    # model = InventoryFairValueModel(
    #     name=f"Inventory_Model_{commodity_key}",
    #     inventory_window=252,
    #     deviation_method='z_score',
    #     # ... other parameters
    # )
    # model_1_configs[commodity_key] = model
    pass

print("Model 1 configurations ready")
```

```
Model 1 configurations ready
```

3.3 Model 2: [Your Second Model]

Implement your second model type.

Model 2 Overview

TODO: Document your second model

Model Type: [e.g., CostOfCarryModel or SupplyDemandModel]

Economic Rationale: [Explain why this model complements Model 1]

Key Features:

1. [Feature 1] - [Why included]
2. [Feature 2] - [Why included]

Model Assumptions:

- [List key assumptions]
- [Differences from Model 1]

In [20]:
```
# TODO: Initialize and configure Model 2
model_2_configs = {}

for commodity_key in COMMODITIES.keys():
    # TODO: Configure model parameters
    pass

print("Model 2 configurations ready")
```

```
Model 2 configurations ready
```

3.4 Model 3 (Optional): [Third Model]

Optional third model for bonus points.

Model 3 Overview (Optional)

Model Type: [Optional third model]

Rationale: [Why add a third model?]

In [21]:
```
# TODO (Optional): Initialize Model 3
model_3_configs = {}

# ... implementation ...
```

3.5 Model Training

Train all models on historical data.

In [22]:
```
# TODO: Train models
def train_models(commodity_key, train_end_date):
    """
    Train all models for a commodity.

    Parameters
    ----------
    commodity_key : str
        Commodity identifier
    train_end_date : datetime or str
        End of training period (as_of_date)
```

```
    Returns
    -------
    dict
        Trained models
    """
    # TODO: Get features and target
    # features = feature_matrices[commodity_key]
    # features = features[features.index <= train_end_date]

    # TODO: Split features and target
    # X = features.drop('price', axis=1)
    # y = features['price']

    # TODO: Train Model 1
    # model_1 = model_1_configs[commodity_key]
    # model_1.fit(X, y, as_of_date=train_end_date)

    # TODO: Train Model 2
    # ...

    # TODO: Train Model 3 (optional)
    # ...

    return {
        'model_1': None,  # TODO
        'model_2': None,  # TODO
        'model_3': None,  # TODO (optional)
    }

print("Model training function ready")
```

```
Model training function ready
```

Part 4: Walk-Forward Validation (20 points)

Validate models using proper walk-forward methodology with no look-ahead bias.

Requirements

1. **Proper fold structure**: Expanding or rolling window
2. **Gap periods**: No overlap between train and test
3. **Strict temporal discipline**: All data queries respect as_of dates
4. **Comprehensive metrics**: RMSE, MAE, directional accuracy, etc.
5. **Model comparison**: Statistical tests across folds

Grading Criteria

- Proper fold structure (8 pts)
- No look-ahead bias (8 pts)
- Comprehensive metrics (4 pts)

4.1 Walk-Forward Setup

In [23]:
```
# TODO: Configure walk-forward validator
wf_validator = WalkForwardValidator(
    initial_train_window=CONFIG['initial_train_window'],
    validation_window=CONFIG['validation_window'],
    test_window=CONFIG['test_window'],
    gap_days=CONFIG['gap_days'],
    window_type='expanding',  # or 'rolling'
)

print("Walk-forward validator configured")
print(f"  Initial train: {CONFIG['initial_train_window']} days")
print(f"  Validation: {CONFIG['validation_window']} days")
print(f"  Test: {CONFIG['test_window']} days")
print(f"  Gap: {CONFIG['gap_days']} days")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[23], line 2
      1 # TODO: Configure walk-forward validator
----> 2 wf_validator = WalkForwardValidator(
      3     initial_train_window=CONFIG[                    ],
      4     validation_window=CONFIG[                    ],
      5     test_window=CONFIG[              ],
      6     gap_days=CONFIG[          ],
      7     window_type=            ,  # or 'rolling'
      8 )
     10 print("Walk-forward validator configured")
     11 print(f"  Initial train: {CONFIG['initial_train_window']} days")

TypeError: WalkForwardValidator.__init__() got an unexpected keyword argument
'initial_train_window'
```

In [24]:
```
# TODO: Create validation folds
def create_validation_folds(start_date, end_date):
    """
    Generate walk-forward validation folds.

    Returns
    -------
    List[dict]
        List of fold definitions with train/validation/test dates
    """
    # TODO: Use WalkForwardValidator.create_folds()
    # folds = wf_validator.create_folds(
    #     start_date=start_date,
    #     end_date=end_date
    # )
    pass

# folds = create_validation_folds(
#     start_date=CONFIG['train_start'],
#     end_date=CONFIG['end_date']
# )
# print(f"Created {len(folds)} validation folds")
```

4.2 Run Walk-Forward Validation

In [25]:
```
# TODO: Implement walk-forward validation loop
validation_results = []
```

```
# for fold_idx, fold in enumerate(folds):
#     print(f"\nFold {fold_idx + 1}/{len(folds)}")
#     print(f"  Train: {fold['train_start']} to {fold['train_end']}")
#     print(f"  Test: {fold['test_start']} to {fold['test_end']}")
#
#     for commodity_key in COMMODITIES.keys():
#         # TODO: Train models on fold training data
#         # models = train_models(commodity_key, fold['train_end'])
#
#         # TODO: Generate predictions on test data
#         # predictions = {}
#         # for model_name, model in models.items():
#         #     pred = model.predict(X_test, as_of_date=fold['test_end'])
#         #     predictions[model_name] = pred
#
#         # TODO: Calculate metrics
#         # metrics = calculate_fold_metrics(predictions, y_test)
#
#         # TODO: Store results
#         # validation_results.append({
#         #     'fold': fold_idx,
#         #     'commodity': commodity_key,
#         #     'metrics': metrics,
#         #     'predictions': predictions,
#         # })
#         pass

print("Walk-forward validation complete")
```

```
Walk-forward validation complete
```

4.3 Validation Metrics

In [26]:

```
# TODO: Calculate comprehensive metrics
def calculate_fold_metrics(predictions, actuals):
    """
    Calculate validation metrics for a fold.

    Parameters
    ----------
    predictions : dict
        Model predictions {model_name: predictions}
    actuals : pd.Series
        Actual values

    Returns
    -------
    dict
        Metrics by model
    """
    from sklearn.metrics import mean_squared_error, mean_absolute_error,

    metrics = {}

    for model_name, preds in predictions.items():
        # TODO: Calculate standard metrics
        # rmse = np.sqrt(mean_squared_error(actuals, preds))
        # mae = mean_absolute_error(actuals, preds)
        # r2 = r2_score(actuals, preds)
```

```
        # TODO: Calculate directional accuracy
        # actual_direction = np.sign(actuals.diff())
        # pred_direction = np.sign(preds.diff())
        # directional_accuracy = (actual_direction == pred_direction).mea

        # TODO: Calculate other metrics
        # - Mean error (bias)
        # - Maximum error
        # - Hit rate at different thresholds

        metrics[model_name] = {
            'rmse': None,  # TODO
            'mae': None,  # TODO
            'r2': None,  # TODO
            'directional_accuracy': None,  # TODO
            'mean_error': None,  # TODO
        }

    return metrics

print("Metrics calculator ready")
```

```
Metrics calculator ready
```

4.4 Model Comparison

In [27]:
```
# TODO: Aggregate and compare model performance
def compare_models(validation_results):
    """
    Compare model performance across all folds.

    Returns
    -------
    pd.DataFrame
        Comparison table with mean and std of metrics
    """
    # TODO: Aggregate metrics across folds
    # Create comparison DataFrame
    # Calculate mean and std for each metric
    # Rank models
    pass

# comparison_df = compare_models(validation_results)
# print(comparison_df)
```

In [28]:
```
# TODO: Visualize model comparison
def plot_model_comparison(validation_results):
    """
    Create visualizations comparing model performance.
    """
    # TODO: Box plots of RMSE by model
    # TODO: Time series of metrics across folds
    # TODO: Scatter plot: predicted vs actual by model
    pass

# plot_model_comparison(validation_results)
```

4.5 Data Leakage Check

Verify no look-ahead bias in validation.

In [29]:
```
# TODO: Run leakage detection
print("Checking for data leakage...\n")

# for fold_idx, fold in enumerate(folds):
#     leakage_report = detect_data_leakage(
#         train_data=...,  # TODO
#         test_data=...,   # TODO
#         train_end_date=fold['train_end'],
#         test_start_date=fold['test_start']
#     )
#
#     if leakage_report['has_leakage']:
#         print(f"⚠️  LEAKAGE DETECTED in fold {fold_idx}!")
#         print(leakage_report)
#     else:
#         print(f"  Fold {fold_idx} clean")

print("\nLeakage check complete")
```

```
Checking for data leakage...


Leakage check complete
```

4.6 Validation Summary

TODO: Summarize validation findings

Validation Results Summary

TODO: Fill in your findings

Performance by Model

Model	RMSE (mean±std)	MAE	R^2	Dir. Accuracy
Model 1	TODO	TODO	TODO	TODO
Model 2	TODO	TODO	TODO	TODO
Model 3	TODO	TODO	TODO	TODO

Best Model by Commodity

Commodity	Best Model	Metric	Value
TODO	TODO	TODO	TODO

Key Findings

1. **Model Performance**: [Which model performed best overall?]
2. **Stability**: [How stable were metrics across folds?]
3. **Commodity Differences**: [Did different models work better for different commodities?]

4. **Temporal Patterns**: [Did performance degrade over time?]

Issues and Solutions

- [Document any validation issues encountered]
- [How did you address them?]

Part 5: Signal Generation (15 points)

Convert fair value estimates to trading signals with proper risk management.

Requirements

1. **Signal logic**: Convert fair value deviation to entry/exit signals
2. **Position sizing**: Size positions based on signal strength
3. **Risk management**: Portfolio-level risk constraints
4. **Performance metrics**: Sharpe, max drawdown, win rate, etc.

Grading Criteria

- Signal logic (6 pts)
- Risk management (5 pts)
- Performance metrics (4 pts)

5.1 Signal Generation Logic

In [30]:
```
# TODO: Configure signal generator
signal_generator = SignalGenerator(
    entry_threshold=CONFIG['entry_threshold'],
    exit_threshold=CONFIG['exit_threshold'],
    lookback_window=252,  # For z-score calculation
    signal_type='mean_reversion',  # or 'momentum'
)

print("Signal generator configured")
print(f"  Entry threshold: {CONFIG['entry_threshold']} std dev")
print(f"  Exit threshold: {CONFIG['exit_threshold']} std dev")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[30], line 2
      1 # TODO: Configure signal generator
----> 2 signal_generator = SignalGenerator(
      3     entry_threshold=CONFIG[                    ],
      4     exit_threshold=CONFIG[                    ],
      5     lookback_window=252,  # For z-score calculation
      6     signal_type=                ,  # or 'momentum'
      7 )
      9 print("Signal generator configured")
     10 print(f"  Entry threshold: {CONFIG['entry_threshold']} std dev")

TypeError: SignalGenerator.__init__() got an unexpected keyword argument
'signal_type'
```

In [31]:

```python
# TODO: Generate signals from fair value estimates
def generate_signals(fair_value, market_price):
    """
    Generate trading signals from fair value vs market price.

    Parameters
    ----------
    fair_value : pd.Series
        Fair value estimates (time series)
    market_price : pd.Series
        Market prices (time series)

    Returns
    -------
    pd.DataFrame
        Signals with columns: date, signal, strength, fair_value, market_
    """
    # TODO: Calculate mispricing
    # mispricing = market_price - fair_value

    # TODO: Normalize mispricing (z-score)
    # z_score = (mispricing - mispricing.rolling(252).mean()) / mispricin

    # TODO: Generate signals
    # Use signal_generator.generate()
    # signals = signal_generator.generate(z_score)

    return None  # TODO

print("Signal generation function ready")
```

```
Signal generation function ready
```

5.2 Position Sizing

In [32]:

```python
# TODO: Configure position sizer
position_sizer = PositionSizer(
    max_position_size=CONFIG['max_position_size'],
    sizing_method='signal_strength',  # or 'equal_weight', 'volatility_ad
    vol_target=0.15,  # 15% annual volatility target
)

print("Position sizer configured")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[32], line 2
      1 # TODO: Configure position sizer
----> 2 position_sizer = PositionSizer(
      3     max_position_size=CONFIG[                    ],
      4     sizing_method=                 ,  # or 'equal_weight',
'volatility_adjusted'
      5     vol_target=0.15,  # 15% annual volatility target
      6 )
      8 print("Position sizer configured")

TypeError: PositionSizer.__init__() got an unexpected keyword argument
'max_position_size'
```

In [33]:
```python
# TODO: Calculate position sizes
def calculate_positions(signals, volatilities):
    """
    Calculate position sizes from signals.

    Parameters
    ----------
    signals : pd.DataFrame
        Trading signals by commodity
    volatilities : pd.Series
        Historical volatilities by commodity

    Returns
    -------
    pd.DataFrame
        Positions (weights) by commodity and date
    """
    # TODO: Use position_sizer.calculate_positions()
    # Consider signal strength, volatility, correlations
    pass

print("Position calculator ready")
```

```
Position calculator ready
```

5.3 Risk Management

In [34]:
```python
# TODO: Configure risk manager
risk_manager = RiskManager(
    max_portfolio_leverage=CONFIG['portfolio_leverage'],
    max_commodity_weight=CONFIG['max_position_size'],
    max_sector_weight=0.6,  # Max 60% in any sector
    risk_budget=0.02,  # 2% daily VaR limit
)

print("Risk manager configured")
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[34], line 2
      1 # TODO: Configure risk manager
----> 2 risk_manager = RiskManager(
      3     max_portfolio_leverage=CONFIG[                    ],
      4     max_commodity_weight=CONFIG[                    ],
      5     max_sector_weight=0.6,  # Max 60% in any sector
      6     risk_budget=0.02,  # 2% daily VaR limit
      7 )
      9 print("Risk manager configured")

TypeError: RiskManager.__init__() got an unexpected keyword argument
'max_portfolio_leverage'
```

In [35]:

```python
# TODO: Apply risk constraints
def apply_risk_constraints(positions, returns_cov):
    """
    Apply portfolio-level risk constraints.

    Parameters
    ----------
    positions : pd.DataFrame
        Target positions before risk constraints
    returns_cov : pd.DataFrame
        Covariance matrix of returns

    Returns
    -------
    pd.DataFrame
        Adjusted positions after risk constraints
    """
    # TODO: Use risk_manager.apply_constraints()
    # Check:
    # - Total leverage
    # - Position concentration
    # - Portfolio VaR
    # - Correlation-adjusted exposure
    pass

print("Risk constraint function ready")
```

```
Risk constraint function ready
```

5.4 Backtesting

In [36]:

```python
# TODO: Run signal backtest
def backtest_signals(signals, positions, prices, costs=0.001):
    """
    Backtest trading signals with transaction costs.

    Parameters
    ----------
    signals : dict
        Signals by commodity
    positions : pd.DataFrame
        Position weights over time
    prices : pd.DataFrame
        Price data by commodity
    costs : float
```

```
        Transaction cost (fraction of notional)

    Returns
    -------
    dict
        Backtest results with returns, metrics, trades
    """
    # TODO: Calculate returns
    # - Position changes (trades)
    # - Transaction costs
    # - Portfolio returns

    # TODO: Calculate metrics
    # - Total return
    # - Sharpe ratio
    # - Max drawdown
    # - Win rate
    # - Average win/loss
    # - Number of trades

    return {}

print("Backtest function ready")
```

```
Backtest function ready
```

In [37]:

```
# TODO: Execute backtest
# backtest_results = backtest_signals(
#     signals=all_signals,
#     positions=final_positions,
#     prices=price_data,
#     costs=0.001
# )
```

5.5 Performance Analysis

In [38]:

```
# TODO: Analyze backtest performance
def analyze_performance(backtest_results):
    """
    Generate comprehensive performance report.
    """
    # TODO: Calculate key metrics
    print("Performance Metrics")
    print("="*50)
    # print(f"Total Return: {total_return:.2%}")
    # print(f"Annualized Return: {ann_return:.2%}")
    # print(f"Sharpe Ratio: {sharpe:.2f}")
    # print(f"Max Drawdown: {max_dd:.2%}")
    # print(f"Win Rate: {win_rate:.2%}")
    # print(f"Profit Factor: {profit_factor:.2f}")

    # TODO: Create performance visualizations
    # - Cumulative returns
    # - Drawdown chart
    # - Rolling Sharpe
    # - Return distribution

    pass
```

```
# analyze_performance(backtest_results)
```

5.6 Signal Summary

TODO: Summarize signal generation and performance

Signal Generation Summary

TODO: Document your signal results

Signal Characteristics

- **Entry Logic**: [Describe when signals are generated]
- **Exit Logic**: [Describe when positions are closed]
- **Average Holding Period**: [TODO days]
- **Trade Frequency**: [TODO trades per year]

Performance Metrics

Metric	Value
Total Return	TODO
Annualized Return	TODO
Sharpe Ratio	TODO
Max Drawdown	TODO
Win Rate	TODO
Profit Factor	TODO

Risk Analysis

- **Portfolio Volatility**: [TODO%]
- **VaR (95%)**: [TODO%]
- **Maximum Position**: [TODO%]

Attribution by Commodity

Commodity	Return	Sharpe	Contribution
TODO	TODO	TODO	TODO

Key Findings

1. **Signal Quality**: [Are signals predictive?]
2. **Risk-Adjusted Returns**: [Is Sharpe attractive?]
3. **Consistency**: [Stable across time?]
4. **Trade-offs**: [Transaction costs vs turnover]

Part 6: Executive Report (5 points)

Professional summary of the project for non-technical stakeholders.

Requirements

1. **Executive Summary** (200 words): High-level overview
2. **Methodology** (300 words): Approach and models
3. **Results** (300 words): Key findings and performance
4. **Conclusions** (200 words): Implications and next steps

Grading Criteria

- Clear communication (3 pts)
- Professional presentation (2 pts)

6.1 Executive Summary

TODO: Write 200-word executive summary

[Write a concise overview covering:

- Project objective
- Approach taken
- Key results
- Bottom-line conclusion

Target audience: Senior management with limited technical background]

6.2 Methodology

TODO: Write 300-word methodology description

[Describe your approach:

- Data sources and quality
- Fair value models selected
- Validation approach
- Signal generation logic

Explain *why* you made key choices, not just *what* you did.]

6.3 Results

TODO: Write 300-word results summary

[Present key findings:

- Model performance comparison
- Validation results
- Signal performance metrics
- Risk characteristics

Use concrete numbers and be specific about what worked and what didn't.]

6.4 Conclusions and Recommendations

TODO: Write 200-word conclusion

[Provide actionable conclusions:

- Is the system ready for deployment?
- What are the main risks?
- What improvements would you prioritize?
- What resources would be needed for production?

Be honest about limitations and uncertainties.]

Final Checklist

Before submitting, verify:

Technical Requirements

- ☐ All code cells execute without errors
- ☐ Point-in-time database created and populated
- ☐ No look-ahead bias (verified with `detect_data_leakage()`)
- ☐ All TODO sections completed
- ☐ Models saved to disk
- ☐ Results exported to files

Documentation Requirements

- ☐ Commodity selection justified
- ☐ Data quality documented
- ☐ Model choices explained
- ☐ Validation approach described
- ☐ Signal logic documented
- ☐ Executive report completed

Visualizations

- ☐ All plots have clear titles
- ☐ Axes labeled with units
- ☐ Legends included where needed
- ☐ Color scheme is readable

Professional Presentation

- ☐ Markdown cells formatted correctly
- ☐ Code is well-commented
- ☐ Consistent naming conventions
- ☐ Output cells show results (not truncated)

Files to Submit

1. This completed notebook: `capstone_project.ipynb`
2. Point-in-time database: `pit_database.db`
3. Saved models: `models/` directory
4. Results and figures: `results/` directory

Congratulations!

You've completed the Fair Value Modelling capstone project. This end-to-end system demonstra

- Robust data infrastructure with point-in-time discipline
- Multiple fair value modeling approaches
- Rigorous walk-forward validation
- Professional signal generation and risk management
- Clear communication of results

These skills are directly applicable to professional quantitative trading and risk management role

Next Steps

Consider extending this project with:

- Additional model types (machine learning, regime-switching)
- Multi-asset portfolio optimization
- Real-time deployment architecture
- Advanced risk models (CVaR, stress testing)
- Transaction cost analysis and optimization

Good luck with your submission!